

INFORME TÉCNICO

Sistema de Créditos IPSFA

Versión del Sistema: v5 (Julio 2026)

Fecha de Análisis: 2026-07-16

Documento generado por análisis automatizado de código fuente

Métrica	Valor
Total de Archivos Analizados	12 archivos fuente
Total de Líneas de Código	~13,000+ líneas
Total de Funciones	~343 funciones
Endpoints API	28+ endpoints
Tiendas Soportadas	3 (Caracas, Maracay, Maracaibo)
Roles de Usuario	2 (Administrador, Operador)

1. ARQUITECTURA GENERAL

1.1 Tecnologías Utilizadas

Capa	Tecnología	Versión
Backend	Node.js + Express	4.x
Base de Datos	PostgreSQL	14+
Driver DB	node-pg (Pool)	8.x
Frontend	HTML5 + CSS3 + Vanilla JS	—
Charts	ApexCharts (CDN)	3.45.1
PDF/Excel	jsPDF + xlsx (CDN)	2.5.1 / 0.18.5
Icons	Font Awesome (CDN)	6.4.0
Auth	JWT (jsonwebtoken)	9.0.2
Validación	express-validator	7.0.1
Hashing	bcryptjs	2.4.3
CORS	cors	2.8.5
Entorno	dotenv	16.3.1

1.2 Estructura de Carpetas

proyecto/

- server.js — Servidor principal Express
- .env — Variables de entorno
- package.json — Dependencias del proyecto
- public/ — Archivos estáticos
- ■■ css/ — panel.css, tienda-caracas.css
- ■■ js/ — panel.js, tienda-caracas-spa.js, tienda-maracay-spa.js, tienda-maracaibo-spa.js
- ■■ assets/ — logo.png, imágenes
- ■■ index.html / login.html — Página de login
- ■■ panel.html — Panel principal (SPA)
- routes/ — Archivos de rutas Express
- ■■ auth.js — Autenticación JWT
- ■■ bcv.js — Tasas BCV (API externa)
- ■■ usuarios.js — Gestión de usuarios
- ■■ actividades.js — Agenda digital
- ■■ estadisticas.js — Estadísticas
- ■■ reportes.js — Reportes filtrados
- config/
- ■■ database.js — Conexión PostgreSQL
- middleware/
- ■■ auth.js — Middleware JWT + auditoría
- uploads/ — Archivos subidos (si aplica)

2. BASE DE DATOS — ESTRUCTURA DE TABLAS

2.1 Tabla: tienda_caracas (y tienda_maracay / tienda_maracaibo — misma estructura)

#	Campo	Tipo	Descripción
1	id	SERIAL PK	ID autoincremental
2	numero	INTEGER	Número correlativo
3	nro_factura	VARCHAR	Número de factura
4	nombre_apellido	VARCHAR	Nombre del cliente
5	monto_factura	DECIMAL	Monto total factura (Bs)
6	fecha_factura	DATE	Fecha de la factura
7	cedula	VARCHAR	Cédula del cliente
8	telefono	VARCHAR	Teléfono del cliente
9	monto_facturado_divisa	DECIMAL	Monto en divisa
10	dolar_facturado	DECIMAL	Monto en dólares
11	cuotas	INTEGER	Número de cuotas
12	monto_pendiente	DECIMAL	Monto pendiente
13	monto_depositados	DECIMAL	Total depositado (calculado)
14	deuda	DECIMAL	Deuda restante (calculado)
15-25	cuota_1 a cuota_11	DECIMAL	Monto de cada cuota
26-36	ref_cuota_1 a ref_cuota_11	VARCHAR	Referencia bancaria
37-47	fecha_cuota_1 a fecha_cuota_11	DATE	Fecha de depósito
48-58	tasa_cuota_1 a tasa_cuota_11	DECIMAL	Tasa BCV del día
59-69	dolar_depositado_cuota_1 a dolar_depositado_cuota_11	DECIMAL	Equivalente en USD
70	updated_at	TIMESTAMP	Última actualización

2.2 Tabla: usuarios

#	Campo	Tipo	Descripción
1	id	SERIAL PK	ID autoincremental
2	nombre	VARCHAR	Nombre completo
3	email	VARCHAR	Correo electrónico
4	password	VARCHAR	Hash de contraseña (bcrypt)
5	rol	VARCHAR	administrador u operador
6	activo	BOOLEAN	Estado activo/inactivo
7	ip_asignada	VARCHAR	IP restringida (operadores)
8	tienda	VARCHAR	Tienda asignada (operadores)
9	fecha_creacion	TIMESTAMP	Fecha de creación
10	fecha_actualizacion	TIMESTAMP	Fecha de última actualización

2.3 Tabla: actividades

#	Campo	Tipo	Descripción
1	id	SERIAL PK	ID autoincremental
2	descripcion	TEXT	Descripción de la actividad
3	descripcion_extra	TEXT	Notas adicionales
4	hora	TIME	Hora programada
5	prioridad	VARCHAR	baja, media, alta, urgente
6	tienda	VARCHAR	caracas, maracay, maracaibo
7	fecha	DATE	Fecha de la actividad
8	estado	VARCHAR	pendiente o completada
9	fecha_completada	TIMESTAMP	Cuándo se completó
10	creado_por	INTEGER	ID del usuario creador
11	completado_por	INTEGER	ID del usuario que completó
12	created_at	TIMESTAMP	Fecha de creación

2.4 Tabla: auditoria_usuarios

#	Campo	Tipo	Descripción
1	id	SERIAL PK	ID autoincremental
2	usuario_id	INTEGER	Usuario afectado
3	usuario_accion_id	INTEGER	Usuario que realizó la acción
4	accion	VARCHAR	Tipo de acción (CREAR, ACTUALIZAR, ELIMINAR, etc.)
5	datos_anteriores	JSONB	Datos antes del cambio
6	datos_nuevos	JSONB	Datos después del cambio
7	ip_address	VARCHAR	IP del usuario
8	created_at	TIMESTAMP	Fecha del evento

2.5 Tabla: bcv_tasas (inferida desde API externa)

#	Campo	Tipo	Descripción
1	id	SERIAL PK	ID autoincremental
2	fecha	DATE	Fecha de la tasa
3	usd	DECIMAL	Tasa USD/VES
4	eur	DECIMAL	Tasa EUR/VES
5	created_at	TIMESTAMP	Fecha de registro

2.6 Tabla: reset_tokens (recuperación de contraseña)

#	Campo	Tipo	Descripción
---	-------	------	-------------

1	id	SERIAL PK	ID autoincremental
2	usuario_id	INTEGER	FK a usuarios
3	token	VARCHAR	Token criptográfico
4	expires_at	TIMESTAMP	Fecha de expiración
5	used	BOOLEAN	¿Ya fue usado?

3. API REST — ENDPOINTS COMPLETOS

3.1 Módulo: Autenticación (/api/auth)

Método	Endpoint	Descripción	Archivo
POST	/api/auth/login	Login con email/password + validación IP	auth.js

Request:

```
{ "email": "usuario@ejemplo.com", "password": "contraseña123" }
```

Response:

```
{ "exito": true, "token": "jwt_token", "usuario": { "id": 1, "nombre": "Admin", "email": "admin@ipsfa.com", "rol": "administrador", "tienda": "caracas" } }
```

3.2 Módulo: BCV (/api/bcv)

Método	Endpoint	Descripción	Archivo
GET	/api/bcv/actual	Tasa actual (USD/EUR)	bcv.js
GET	/api/bcv/anterior	Tasa anterior	bcv.js
GET	/api/bcv/fechas	Listado de fechas disponibles	bcv.js
GET	/api/bcv/fecha/:fecha	Tasa por fecha específica + fallback	bcv.js + server.js
GET	/api/bcv/historial/:year	Historial anual	bcv.js

Response típico:

```
{ "exito": true, "tasa": { "current": { "usd": 721.3456, "eur": 785.4321, "date": "2026-07-14" }, "previous": { "usd": 720.1234, "eur": 784.5678, "date": "2026-07-13" }, "changePercentage": { "usd": 0.17, "eur": 0.11 } } }
```

3.3 Módulo: Usuarios (/api/usuarios)

Método	Endpoint	Descripción	Archivo
GET	/api/usuarios	Listar usuarios (con filtros)	usuarios.js
GET	/api/usuarios/:id	Obtener usuario por ID	usuarios.js
POST	/api/usuarios	Crear usuario (IP obligatoria para operadores)	usuarios.js
PUT	/api/usuarios/:id	Actualizar usuario	usuarios.js
DELETE	/api/usuarios/:id	Desactivar usuario (soft delete)	usuarios.js
PATCH	/api/usuarios/:id/reactivar	Reactivar usuario	usuarios.js
PUT	/api/usuarios/:id/password	Cambiar contraseña	usuarios.js
POST	/api/usuarios/recuperar-password	Solicitar recuperación	usuarios.js
POST	/api/usuarios/restablecer-password	Restablecer con token	usuarios.js
GET	/api/usuarios/perfil/me	Perfil del usuario logueado	usuarios.js
GET	/api/usuarios/auditoria/:usuariold	Historial de auditoría	usuarios.js
GET	/api/usuarios/estadisticas/resumen	Estadísticas de usuarios	usuarios.js

Query params para GET /api/usuarios:

- `busqueda` — Filtrar por nombre/email
- `rol` — administrador u operador
- `activo` — true o false
- `ordenar_por` — nombre, email, created_at, updated_at
- `orden` — ASC o DESC

3.4 Módulo: Tienda Caracas (/api/tienda-caracas) — INLINE en server.js

Método	Endpoint	Descripción
GET	/api/tienda-caracas	Listar todos los clientes
GET	/api/tienda-caracas/:id	Obtener cliente por ID
POST	/api/tienda-caracas	Crear nuevo cliente + cuota 1
PUT	/api/tienda-caracas/:id	Actualizar cliente (cuotas, teléfono)
DELETE	/api/tienda-caracas/:id	Eliminar cliente

3.5 Módulo: Tienda Maracay (/api/tienda-maracay) — INLINE en server.js

Método	Endpoint	Descripción
GET	/api/tienda-maracay	Listar todos los clientes
GET	/api/tienda-maracay/:id	Obtener cliente por ID
POST	/api/tienda-maracay	Crear nuevo cliente + cuota 1
PUT	/api/tienda-maracay/:id	Actualizar cliente (cuotas, teléfono)
DELETE	/api/tienda-maracay/:id	Eliminar cliente

3.6 Módulo: Tienda Maracaibo (/api/tienda-maracaibo) — INLINE en server.js

Método	Endpoint	Descripción
GET	/api/tienda-maracaibo	Listar todos los clientes
GET	/api/tienda-maracaibo/:id	Obtener cliente por ID
POST	/api/tienda-maracaibo	Crear nuevo cliente + cuota 1
PUT	/api/tienda-maracaibo/:id	Actualizar cliente (cuotas, teléfono)
DELETE	/api/tienda-maracaibo/:id	Eliminar cliente

3.7 Módulo: Actividades (/api/actividades)

Método	Endpoint	Descripción	Archivo
GET	/api/actividades	Listar actividades (filtros: fecha, tienda, estado)	actividades.js
GET	/api/actividades/:id	Obtener actividad por ID	actividades.js
POST	/api/actividades	Crear actividad	actividades.js
PUT	/api/actividades/:id	Actualizar actividad (estado, descripción)	actividades.js
DELETE	/api/actividades/:id	Eliminar actividad	actividades.js
GET	/api/actividades/estadisticas/resumen	Resumen estadístico	actividades.js

Query params para GET /api/actividades:

- fecha — Filtrar por fecha (YYYY-MM-DD)
- fecha_desde / fecha_hasta — Rango de fechas
- tienda — caracas, maracay, maracaibo
- estado — pendiente o completada

3.8 Módulo: Estadísticas (/api/estadisticas)

Método	Endpoint	Descripción	Archivo
GET	/api/estadisticas	Resumen general (KPIs)	estadisticas.js
GET	/api/estadisticas/deudores	Listado de deudores	estadisticas.js
GET	/api/estadisticas/evolucion	Evolución mensual	estadisticas.js
GET	/api/estadisticas/distribucion	Distribución de pagos	estadisticas.js
GET	/api/estadisticas/exportar	Exportar datos	estadisticas.js

3.9 Módulo: Reportes (/api/reportes)

Método	Endpoint	Descripción	Archivo
POST	/api/reportes/caracas	Generar reporte filtrado Caracas	reportes.js
POST	/api/reportes/maracaibo	Generar reporte filtrado Maracaibo	reportes.js

Body del POST /api/reportes/caracas:

```
{"fecha_desde": "2026-01-01", "fecha_hasta": "2026-07-14", "estado": "todos", "monto_min": 0, "monto_max": 9999999, "nombre_cliente": "", "cedula": ""}
```

Response:

```
{"exito": true, "datos": [...], "resumen": {"total_clientes": N, "total_deuda": N, "total_depositado": N, "clientes_mora": N, "promedio_deuda": N}}
```

4. FRONTEND — ESTRUCTURA DE ARCHIVOS

4.1 Archivo: panel.html (SPA Principal)

Rol: Página principal del sistema. Single Page Application (SPA) que carga todos los módulos.

Secciones (content-area):

ID	Nombre	Descripción
contentDashboard	Panel Principal	Reloj, tasa BCV, widget actividades, KPIs
contentTasas	Tasas BCV	Consulta por fecha + historial anual
contentUsuarios	Gestión de Usuarios	CRUD de usuarios, estadísticas, filtros, auditoría
contentPerfil	Mi Perfil	Datos personales, cambio de contraseña
contentClientes	Tienda Caracas	Módulo completo Caracas (menú → base datos → conciliaciones → reportes)
contentCreditos	Tienda Maracay	Módulo completo Maracay (misma estructura que Caracas)
contentPagos	Tienda Maracaibo	Módulo completo Maracaibo (misma estructura que Caracas)
contentReportes	Actividades Pendientes	Agenda digital con CRUD de actividades + historial paginado
contentEstadisticas	Estadísticas	KPIs, gráficos ApexCharts, tabla de deudores, filtros por tienda

Navegación Sidebar (nav-item):

- nav-dashboard → Panel Principal
- nav-caracas → Tienda Caracas
- nav-maracay → Tienda Maracay
- nav-maracaibo → Tienda Maracaibo
- nav-tasas → Tasas BCV
- nav-usuarios → Usuarios
- nav-reportes → Actividades Pendientes

Modales:

- modalUsuario / modalUsuarioTitulo — Crear/Editar usuario
- modalConfirmar — Confirmación de acciones
- modalAuditoria — Historial de auditoría
- modalFecha / modalFechaInput / modalResultado — Consulta tasa por fecha
- modal-corporativo — Modal genérico reutilizable (icono, título, mensaje, botones personalizados)
- modal-editar-actividad — Edición inline de actividades
- modal-descripcion-widget — Añadir descripción a actividad desde widget

Scripts cargados:

```
<script src="js/panel.js"></script>
<script src="js/tienda-caracas-spa.js"></script>
<script src="js/tienda-maracay-spa.js"></script>
<script src="js/tienda-maracaibo-spa.js"></script>
<script src="https://cdn.jsdelivr.net/npm/apexcharts@3.45.1/dist/apexcharts.min.js"></script>
<script src="https://cdnjs.cloudflare.com/ajax/libs/jspdf/2.5.1/jspdf.umd.min.js"></script>
<script src="https://cdnjs.cloudflare.com/ajax/libs/jspdf-autotable/3.5.31/jspdf.plugin.autotable.min.js"></script>
<script src="https://cdnjs.cloudflare.com/ajax/libs/xlsx/0.18.5/xlsx.full.min.js"></script>
```

4.2 Archivo: panel.js (Lógica Principal SPA)

Tamaño: ~3,500 líneas | **Funciones:** 94 declaradas | **Scope:** Global en panel.html

Variables Globales:

```
const token = localStorage.getItem('token');
let usuario = {};
let usuariosData = [];
let usuarioEditando = null;
let chartEvolucion = null;
let chartDistribucion = null;
let accionPendiente = null;
let usuarioPendiente = null;
let actividades = [];
let paginaHistorial = 1;
let totalPaginasHistorial = 1;
let actividadesHistorialCache = [];
let datosBusquedaCaracas = [];
let resumenBusquedaCaracas = {};
let paginaBusqueda = 1;
let registrosPorPaginaBusqueda = 10;
let totalPaginasBusqueda = 1;
let datosBusquedaMaracaibo = [];
let resumenBusquedaMaracaibo = {};
let paginaBusquedaMB = 1;
let registrosPorPaginaBusquedaMB = 10;
let totalPaginasBusquedaMB = 1;
```

Secciones principales del código:

Sección	Funciones Clave	Descripción
Autenticación	sincronizarPerfilUsuario(), cerrarSesion()	Login, logout, verificación de token, sincronización de perfil desde backend
Sidebar	toggleSidebar(), mostrarSeccion()	Navegación entre secciones, protección por rol y tienda
Reloj	actualizarReloj()	Reloj en tiempo real, calendario semanal
Tasa BCV	cargarTasaActual(), consultarTasaFecha(), cargarHistorial()	Consulta de tasas, historial anual, fallback por fecha
Usuarios	cargarUsuarios(), renderizarUsuarios(), guardarUsuario(), editarUsuario(), eliminarUsuario(), sincronizarPerfiles()	CRUD de Usuarios, gestión de permisos, sincronización IP
Perfil	cargarPerfil(), formCambiarPassword	Datos personales, cambio de password
Estadísticas	initEstadisticas(), cargarDatosEstadisticasReales(), procesarDatosEstadisticasReales(), actualizarKpisReales(), filtrarPorPeriodo()	Dashboard de KPIs Reales, filtros por periodo
Agenda	initAgenda(), agregarActividad(), toggleActividad(), eliminarActividad(), editarActividad(), cargarActividadesPaginado()	CRUD de actividades, calendario de actividades paginado
Widget Actividades	actualizarWidgetActividades(), completarActividadDesdeWidget(), abrirModalDetallesWidget(), guardarDescripcionWidget()	Widget de actividades, gestión de detalles y descripción
Reportes Caracas	mostrarBusqueda(), generarBusquedaCaracas(), renderizarTablaBusqueda(), exportarBusquedaExcel(), exportarBusquedaPDF()	Reportes de búsquedas, filtros, exportación
Reportes Maracaibo	mostrarBusquedaMaracaibo(), generarBusquedaMaracaibo(), renderizarTablaBusquedaMB(), exportarBusquedaExcelMaracaibo(), exportarBusquedaPDFMaracaibo()	Reportes de búsquedas, filtros, exportación Maracaibo
Responsive	esMovil(), ajustarVistaMovil()	Detección móvil, adaptación de UI
Control de Acceso	isAdmin(), getUserRole(), getTiendaUsuario(), ocultarMenuSegunRol()	Roles: admin (todo) vs operador (solo su tienda)

Flujo de navegación:

- Usuario hace clic en nav-item → mostrarSeccion("clientes")
- Se ocultan todas las content-area y se muestra la seleccionada
- Se verifica permiso según rol (isAdmin() / getTiendaUsuario())
- Se ejecuta función de inicialización específica (ej: initTiendaCaracas())
- Se cargan datos desde API

4.3 Archivo: tienda-caracas-spa.js (Módulo SPA Caracas)

Tamaño: ~1,500 líneas | Funciones: 61 | Scope: Dentro de panel.html (sección contentClientes)

Variables Globales (dentro del script):

```
const API_BASE_URL = window.location.origin + '/api';
const ITEMS_PER_PAGE_DEFAULT = 25;
let allData = [];
let filteredData = [];
let currentPage = 1;
let itemsPerPage = 25;
let currentFilter = 'all';
let isInitialized = false;
let currentEditId = null;
let currentEditItem = null;
let conciliacionClienteActual = null;
let conciliacionTasaActual = null;
let cuotasAEliminar = [];
```

Sub-módulos dentro de Caracas:

Sub-módulo	Funciones Clave	Descripción
Base de Datos	initTiendaCaracas(), loadData(), renderTable(), applyFilters(), updateSummary()	Tabla paginada con filtros rápidos y avanzados
Modal Edición	verDetalle(), createModalElement(), fillFormData(), guardarCambios(), closeModal()	Ver detalle cliente, cuotas, teléfono. Admin=edita, Operador=solo lee
Eliminar Cuotas	confirmarEliminarCuotas(), ejecutarEliminarCuotas()	Checklist de cuotas a eliminar, recálculo de deuda
Conciliaciones	mostrarConciliaciones(), buscarFacturaConciliacion(), mostrarClienteEncontrado(), mostrarFacturaNoEncontrada(), calcularDeudaConciliacion(), verificarDeuda=0	Busca el número de factura a conciliar
Nuevo Registro	mostrarFormularioNuevoRegistro(), guardarNuevaConciliacion()	Crear cliente + cuota 1
Tasa BCV	obtenerTasaPorFechaConciliacion(), obtenerTasaNuevaConciliacion(), calcularDolarDepositado(), fallback a actual, fallback hardcoded	Consultar tasa de conciliación
Exportación	exportToExcel(), exportToPDF(), printTable()	CSV, PDF con jsPDF, impresión
Modal Corporativo	mostrarModalCorporativo(), cerrarModalCorporativo()	Modal reutilizable con icono, título, mensaje, botones personalizados
Eliminar Cliente	confirmarEliminarCliente(), eliminarCliente()	Confirmación + eliminación API

Flujo de Base de Datos:

- initTiendaCaracas() → carga datos desde /api/tienda-caracas
- processItemData() → calcula: monto_depositados (suma cuotas), deuda (monto_factura - depositado)
- renderTable() → muestra tabla paginada con: N°, Factura, Cliente, Monto, Fecha, Cédula, Cuotas, Depositado, Deuda, Estado, Acciones
- applyFilters() → filtra por: búsqueda general, factura, cédula, fechas, montos, estado (todos/deudores/incompletos/al día/abiertas/canceladas)

Flujo de Conciliaciones:

- Usuario ingresa N° de factura → buscarFacturaConciliacion()
- Si existe: muestra info + historial de cuotas + formulario para nueva cuota
- Si deuda = 0: modal "Factura Cancelada" → opción de agregar cuota adicional
- Si no existe: modal "Factura No Encontrada" → opción de crear nuevo registro
- Formulario cuota: monto Bs → consulta tasa BCV → calcula dólar depositado → guarda

4.4 Archivo: tienda-maracay-spa.js (Módulo SPA Maracay)

Tamaño: ~1,500 líneas | Funciones: 61 | Scope: Dentro de panel.html (sección contentCreditos)

Estructura **IDÉNTICA** a tienda-caracas-spa.js pero:

- Variables terminan en M (ej: allDataM, currentPageM, initTiendaMaracay())
- API endpoint: /api/tienda-maracay en lugar de /api/tienda-caracas

- Envuelto en IIFE (function(){...})() para aislar variables
- Mismo flujo, mismas funciones, mismas reglas de negocio

4.5 Archivo: tienda-maracaibo-spa.js (Módulo SPA Maracaibo)

Tamaño: ~1,500 líneas | **Funciones:** 61 | **Scope:** Dentro de panel.html (sección contentPagos)

Estructura **IDÉNTICA** a tienda-caracas-spa.js pero:

- Variables terminan en MB (ej: allDataMB, currentPageMB, initTiendaMaracaibo())
- API endpoint: /api/tienda-maracaibo en lugar de /api/tienda-caracas
- Envuelto en IIFE (function(){...})() para aislar variables
- Exposición de funciones al window global mediante objeto window
- Variable window.tiendaActiva para integración con estadísticas

5. ROLES Y PERMISOS

5.1 Administrador (rol: 'administrador')

- Acceso a **TODAS** las secciones
- CRUD completo de usuarios (crear, editar, desactivar, reactivar, auditoría)
- Editar cuotas de clientes (todos los campos editables)
- Eliminar cuotas individuales (checklist + recálculo automático)
- Ver auditoría de usuarios
- Ver actividades de **TODAS** las tiendas
- Estadísticas globales (puede seleccionar cualquier tienda)
- Reportes de todas las tiendas
- Cambiar contraseña de cualquier usuario
- Asignar/quitar IP restringida a operadores

5.2 Operador (rol: 'operador')

- Solo su tienda asignada (tienda: 'caracas', 'maracay' o 'maracaibo')
- Ver clientes de su tienda (solo lectura, excepto teléfono editable)
- Agregar cuotas (conciliaciones)
- Crear nuevos clientes
- Ver y crear actividades de su tienda
- **NO** puede ver usuarios
- **NO** puede ver actividades de otras tiendas
- **NO** puede editar cuotas existentes
- **NO** puede eliminar cuotas
- **NO** puede acceder a estadísticas globales
- **NO** puede acceder a reportes de otras tiendas
- Acceso restringido por **IP asignada** (validación en login)

5.3 Control de Acceso en Código

Backend (auth.js — validación IP en login):

```
// Operadores DEBEN tener IP asignada y coincidir
if (usuario.rol === 'operador') {
  if (!usuario.ip_asignada) → 403 "Sin IP asignada"
  if (usuario.ip_asignada !== ipCliente) → 403 "IP no autorizada"
}
// Administradores: si tienen IP, validarla; si no, permitir
if (usuario.rol === 'administrador' && usuario.ip_asignada) {
  if (usuario.ip_asignada !== ipCliente) → 403
}
```

Frontend (panel.js — mostrarSeccion()):

```
if (!isAdmin()) {
  const tiendaUsuario = getTiendaUsuario();
  // Actividades Pendientes: solo admin
  if (seccion === 'reportes') { mostrarAlerta('No tienes permiso...'); return; }
  // Tiendas: solo la que le corresponde al operador
  const tiendaMap = {'clientes':'caracas', 'creditos':'maracay', 'pagos':'maracaibo'};
  if (tiendaMap[seccion] !== tiendaUsuario) { mostrarAlerta('No tienes permiso...'); return; }
  // Usuarios: solo admin
  if (seccion === 'usuarios') { mostrarAlerta('No tienes permiso...'); return; }
}
```

Frontend (tienda-caracas-spa.js — Modal de edición):

```
// El modal se crea DINÁMICAMENTE según rol:  
// Admin: inputs editables para cuotas  
// Operador: divs readonly (solo teléfono editable)  
const isAdminUser = isAdmin();  
const modalTitle = isAdminUser ? 'Editar Cliente' : 'Ver Cliente';  
const cuotasTitle = isAdminUser ? 'Cuotas (Editables)' : 'Cuotas (Solo Lectura)';
```

6. FLUJOS DE TRABAJO CLAVE

6.1 Flujo: Crear Cliente + Primera Cuota (Conciliaciones)

1. Usuario va a Tienda Caracas → Conciliaciones Bancarias
2. Ingresa N° de factura → buscarFacturaConciliacion()
3. Si NO existe → "Factura No Encontrada" → "Registrar Nuevo Cliente"

4. Formulario:

- N° Factura (readonly, del input anterior)
- Fecha Factura
- Nombre y Apellido
- Cédula
- Monto Factura (Bs)
- Primera Cuota:
 - Monto (Bs) → oninput: calcularDolarNuevaConciliacion()
 - Referencia
 - Fecha Depósito → onchange: obtenerTasaNuevaConciliacion()
 - Tasa BCV → calculada automáticamente
 - Dólar Depositado → calculado (monto / tasa)

5. Guardar → POST /api/tienda-caracas

Body: {nro_factura, nombre_apellido, cedula, monto_factura, fecha_factura, monto_depositados, deuda, cuota_1, ref_cuota_1, fecha_cuota_1, tasa_cuota_1, dolar_depositado_cuota_1}

6.2 Flujo: Agregar Cuota a Cliente Existente

1. Usuario va a Conciliaciones → ingresa N° factura
2. Si existe → muestra info del cliente + historial de cuotas + formulario para nueva cuota
3. Formulario nueva cuota:

- N° Cuota (auto: cuotas_pagadas + 1)
- Monto (Bs)
- Referencia
- Fecha Depósito → consulta tasa BCV
- Tasa BCV
- Dólar Depositado

4. Guardar → PUT /api/tienda-caracas/:id

Body: {cuota_X, ref_cuota_X, fecha_cuota_X, tasa_cuota_X, dolar_depositado_cuota_X, monto_depositados, deuda}

5. Si deuda después = 0 → modal "Factura Cancelada" → opción de agregar cuota adicional

6.3 Flujo: Editar Cliente (Admin)

1. Base de Datos → clic en "Ver y editar" (ojo)
2. Modal se crea DINÁMICAMENTE según rol:
 - Admin: inputs editables para cuotas
 - Operador: divs readonly (solo teléfono editable)
3. Se muestran SOLO las cuotas que existen en BD (cuota_X > 0)
4. Admin puede editar: monto, referencia, fecha, tasa de cada cuota
5. Sección "Eliminar Cuotas" con checklist (solo admin)

6. Guardar → PUT /api/tienda-caracas/:id

- Recalcula monto_depositados y deuda
- Preserva monto_factura original (CRÍTICO: no lo lee del input formateado)
- CRITICAL FIX: monto_factura viene del item original, NO del input formateado. El input usa formatNumber() que

convierte 39035.88 → "39.035,88". Si se lee del input, `parseFloatES("39.035,88")` → 39.035 (¡pierde 88% del valor!)

6.4 Flujo: Eliminar Cuotas (Admin)

1. En modal de edición, sección "Eliminar Cuotas"
2. Admin marca checkboxes de cuotas a eliminar
3. Clic en "Borrar Cuotas Seleccionadas" → `confirmarEliminarCuotas()`
4. Modal de confirmación muestra detalle de cuotas a eliminar
5. `ejecutarEliminarCuotas()` → `PUT /api/tienda-caracas/:id`
 - Setea a null: `cuota_X`, `ref_cuota_X`, `fecha_cuota_X`, `tasa_cuota_X`, `dolar_depositado_cuota_X`
 - Recalcula `monto_depositados` (suma cuotas restantes)
 - Recalcula `deuda` = `monto_factura` - `monto_depositados`
 - Actualiza datos locales y refresca tabla

6.5 Flujo: Reportes Caracas

1. Tienda Caracas → Reportes
2. Filtros: `fecha_desde`, `fecha_hasta`, `estado`, `monto_min`, `monto_max`, `nombre_cliente`
3. Generar → `POST /api/reportes/caracas`
4. Respuesta: `{exito, datos[], resumen {total_clientes, total_deuda, total_depositado, clientes_mora, promedio_deuda}}`
5. Muestra:
 - Cards resumen (5 KPIs)
 - Tabla paginada (10/25/50/100 registros por página)
 - Gráficos: barras (deuda por estado) + pastel (distribución)
 - Botones exportar Excel y PDF
6. Exportar PDF: jsPDF con logo, encabezado corporativo, tabla de datos, leyenda de sumatorias al final

7. REGLAS DE NEGOCIO CRÍTICAS

7.1 Cálculos Financieros

processItemData() — aplicado en frontend y backend:

```
montoDepositado = suma(cuota_1 ... cuota_11) donde cuota_X > 0
deuda = monto_factura - montoDepositado
if (abs(monto_factura - montoDepositado) < 0.01) deuda = 0
```

Estados del cliente:

Condición	Estado	Clase CSS	Descripción
deuda <= 0	cancelada	badge-success	Factura pagada completamente
deuda > 0 && cuotas_pagadas === 0	abierta	badge-info	Factura registrada, sin pagos
deuda > 0 && cuotas_pagadas > 0 && cuotas_pagadas < total_cuotas	pagada parcial	badge-warning	Pagos parciales
deuda > 0 && cuotas_pagadas >= total_cuotas	al día	badge-success	Cuotas completadas pero deuda > 0
deuda > 0 (general)	deudor	badge-danger	Tiene deuda pendiente

7.2 Parseo de Números (Formato Español/Venezolano)

parseNumberES() — CRÍTICO para no perder decimales:

```
// Ej: "39.035,88" → 39035.88 (punto = miles, coma = decimal)
// Ej: "1234.56" → 1234.56 (formato inglés, sin miles)
function parseNumberES(value) {
  if (typeof value === 'number') return value;
  let str = String(value).trim();
  const lastComma = str.lastIndexOf(',');
  const lastDot = str.lastIndexOf('.');
  if (lastComma > lastDot) {
    // Formato español: 1.234,56 → 1234.56
    cleaned = str.replace(/\.\/g, '').replace(',', '.');
  } else {
    // Formato inglés o sin separador
    cleaned = str.replace(/,/g, '');
  }
  return parseFloat(cleaned);
}
```

■ **BUG HISTÓRICO CORREGIDO:** Valores como "39.035,88" se parseaban como 39.035 (perdía .88). Solución: detectar último separador.

7.3 Formateo de Moneda

formatCurrency() — muestra Bs con separador español:

```
// Ej: 39035.88 → "39.035,88 Bs"
function formatCurrency(value) {
  const num = parseFloat(value);
  return new Intl.NumberFormat('es-VE', {
    minimumFractionDigits: num % 1 === 0 ? 0 : 2,
    maximumFractionDigits: 2
  }).format(num) + ' Bs';
}
```

7.4 Formateo de Números

formatNumber() — separador de miles español:

```
// Ej: 39035.88 → "39.035,88"  
// Ej: 1234 → "1.234"  
function formatNumber(value) {  
  const num = parseFloat(value);  
  return new Intl.NumberFormat('es-VE', {  
    minimumFractionDigits: 0,  
    maximumFractionDigits: 2  
  }).format(num);  
}
```

7.5 Cálculo de Dólar Depositado

En conciliaciones:

```
dolar_depositado = monto_bs / tasa_bcv  
// Ej: 10000 Bs / 721.3456 = 13.86 USD  
// Redondeo a 2 decimales  
const dolar = (parseFloat(montoBs) / parseFloat(tasa)).toFixed(2);
```

7.6 Recálculo Automático en Edición

Al guardar cambios (PUT):

1. Leer monto_factura del ítem original (NO del input formateado)
2. Sumar todas las cuotas_X > 0 → monto_depositados
3. deuda = monto_factura - monto_depositados
4. Si deuda < 0.01 → deuda = 0 (evitar valores negativos por redondeo)
5. Incluir monto_depositados y deuda en el body del PUT
6. Backend recalcula y actualiza updated_at

8. CONFIGURACIÓN Y VARIABLES DE ENTORNO

8.1 .env (Requerido)

```
DB_HOST=localhost
DB_PORT=5432
DB_NAME=creditos
DB_USER=postgres
DB_PASSWORD=admin123
PORT=3000
JWT_SECRET=ipsfa_secret_key_2024
JWT_EXPIRES_IN=24h
```

8.2 Configuración en server.js

Pool de conexión PostgreSQL:

```
const pool = new Pool({
  host: process.env.DB_HOST || 'localhost',
  port: process.env.DB_PORT || 5432,
  database: process.env.DB_NAME || 'creditos',
  user: process.env.DB_USER || 'postgres',
  password: process.env.DB_PASSWORD || 'admin123'
});
```

Content Security Policy (CSP):

```
app.use((req, res, next) => {
  res.setHeader('Content-Security-Policy',
    "default-src 'self'; " +
    "script-src 'self' 'unsafe-inline' 'unsafe-eval' https://cdn.jsdelivr.net https://cdnjs.cloudflare.com; "
+
    "style-src 'self' 'unsafe-inline' https://cdnjs.cloudflare.com https://cdn.jsdelivr.net; " +
    "img-src 'self' data: blob; " +
    "font-src 'self' https://cdnjs.cloudflare.com; " +
    "connect-src 'self' http://localhost:* https://localhost:* https://cdnjs.cloudflare.com
https://cdn.jsdelivr.net;")
  );
  next();
});
```

8.3 Middleware JWT (routes/auth.js)

Verificación de token:

```
const authenticateToken = (req, res, next) => {
  const token = req.headers['authorization']?.split(' ')[1];
  if (!token) return res.status(401).json({ error: 'Token requerido' });
  jwt.verify(token, process.env.JWT_SECRET, (err, user) => {
    if (err) return res.status(403).json({ error: 'Token inválido' });
    req.user = user;
    next();
  });
};
```

8.4 Middleware de Auditoría (routes/auth.js)

Registro automático de cambios:

```
const auditMiddleware = (accion) => async (req, res, next) => {
  const originalJson = res.json.bind(res);
  res.json = async (body) => {
    if (body.exito) {
      await pool.query(
        'INSERT INTO auditoria_usuarios (...) VALUES (...)',
        [req.user.id, req.user.id, accion,
datosAnteriores, body.data, req.ip]
```

```
    );  
  }  
  originalJson(body);  
};  
next();  
};
```

9. PUNTOS DE ATENCIÓN Y POSIBLES PROBLEMAS

9.1 Problemas Conocidos

#	Problema	Ubicación	Impacto	Solución Temporal
1	BCV fallback hardcodeado	bcv.js + server.js	Tasa fija 721.3456 si API BCV falla	Revisar bcv.js para conexión real
2	Duplicación de código	tienda-caracas-spa.js vs tienda-maracay-spa.js	Mayor tiempo de desarrollo	Refactorizar a módulo genérico
3	API_BASE_URL hardcodeado en tienda-caracas.js	tienda-caracas.js (standalone)	No funciona en producción	Cambiar a window.location.origin
4	Sin manejo de errores de red	Varios fetch en frontend	App se cuelga si backend cae	Agregar try/catch en todos los fetch
5	No hay validación de token expirado	panel.js	401 no redirige a login	Agregar interceptor de fetch
6	Tienda Maracaibo no tenía módulo SPA	panel.html (placeholder)	Era placeholder vacío	■ DESARROLLADO en v5 — módulo completo
7	Datos de ejemplo en tienda-caracas.js	tienda-caracas.js (standalone)	Muestra datos falsos si API falla	Eliminar o hacer opcional
8	Falta índice en búsqueda de factura	server.js	Búsqueda O(n) en todos los registros	Agregar índice DB en nro_factura
9	No hay paginación en backend	server.js (GET /api/tienda-caracas)	Carga TODOS los registros	Implementar LIMIT/OFFSET
10	CSS duplicado	panel.css + tienda-caracas.css	Conflicto de estilos posible	Unificar CSS o usar scoped
11	IP de operador no se valida en frontend	panel.js	Solo se valida en login	Agregar validación periódica
12	Auditoría solo registra éxitos	routes/auth.js	Fallos no se auditan	Extender a fallos críticos
13	No hay rate limiting	server.js	Vulnerable a ataques de fuerza bruta	Implementar express-rate-limit

9.2 Dependencias CDN (Sin package.json local)

Librería	URL CDN	Versión	Uso
ApexCharts	https://cdn.jsdelivr.net/npm/apexcharts@3.45.1	3.45.1	Gráficos de estadísticas
jsPDF	https://cdnjs.cloudflare.com/ajax/libs/jspdf/2.5.1	2.5.1	Exportación a PDF
jsPDF-autotable	https://cdnjs.cloudflare.com/ajax/libs/jspdf-autotable/3.5.31	3.5.31	Tablas en PDF
xlsx	https://cdnjs.cloudflare.com/ajax/libs/xlsx/0.18.5	0.18.5	Exportación a Excel
Font Awesome	https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0	6.4.0	Iconografía

10. GLOSARIO DE FUNCIONES POR ARCHIVO

10.1 panel.js — Funciones Principales (94 funciones)

Autenticación y Sesión:

- sincronizarPerfilUsuario() — Sincroniza datos del usuario desde API (GET /api/usuarios/perfil/me)
- cerrarSesion() — Limpia localStorage y redirige a login
- verificarTokenExpirado() — Verifica si el token JWT ha expirado

Navegación:

- mostrarSeccion(seccion) — Muestra sección, verifica permisos, ejecuta init
- toggleSidebar() — Abre/cierra sidebar (móvil/desktop)
- esMovil() — Detecta si es dispositivo móvil
- ajustarVistaMovil() — Adapta UI para móvil

Reloj:

- actualizarReloj() — Actualiza reloj, fecha, calendario semanal

Tasa BCV:

- cargarTasaActual() — Carga tasa actual desde API
- consultarTasaFecha() — Abre modal para consultar fecha
- buscarTasaModal() — Consulta tasa específica
- cargarHistorial() — Carga historial de tasas anual

Usuarios:

- cargarUsuarios() — GET /api/usuarios con filtros
- renderizarUsuarios() — Renderiza tabla de usuarios
- filtrarUsuarios() — Aplica filtros de búsqueda
- abrirModalUsuario() — Abre modal crear/editar
- guardarUsuario(event) — POST/PUT usuario con validación
- editarUsuario(id) — Carga usuario para edición
- confirmarEliminar(id, nombre) — Modal confirmar desactivar
- confirmarReactivar(id, nombre) — Modal confirmar reactivar
- ejecutarAccionPendiente() — Ejecuta acción confirmada
- cargarEstadisticasUsuarios() — Carga contadores de usuarios
- cargarAuditoriaUsuario(id) — Carga historial de auditoría
- renderizarAuditoria() — Renderiza tabla de auditoría

Perfil:

- cargarPerfil() — Carga datos del perfil logueado
- formCambiarPassword (event listener) — Cambio de contraseña con validación

Estadísticas:

- initEstadisticas() — Inicializa gráficos ApexCharts
- cargarDatosEstadisticasReales() — Carga datos desde API
- procesarDatosEstadisticas(clientes, mes, anio, tipo) — Procesa datos crudos
- actualizarKPIsReales(kpis) — Actualiza cards de KPIs
- actualizarGraficosReales(evolucion, distribucion) — Actualiza charts
- actualizarTablaDeudoresReales(deudores) — Renderiza tabla deudores
- cargarMesesDisponibles() — Llena dropdown de meses con datos
- inicializarGraficos() — Crea instancias ApexCharts
- aplicarFiltros() — Aplica filtros y recarga datos
- cambiarTiendaEstadisticas() — Cambia tienda en estadísticas (admin)
- exportarEstadisticasPDF() — Exporta estadísticas a PDF

Agenda (Actividades):

- `initAgenda()` — Inicializa agenda del día
- `agregarActividad()` — Crea nueva actividad
- `toggleActividad(id)` — Marca/desmarca como completada
- `eliminarActividad(id)` — Elimina actividad con confirmación
- `editarActividad(id)` — Edita descripción de actividad
- `filtrarActividades(filtro)` — Filtra por todas/pendientes/completadas
- `renderizarActividades()` — Renderiza lista de actividades
- `actualizarStatsAgenda()` — Actualiza contadores (total/pendientes/completadas)
- `cargarHistorialActividades()` — Carga historial paginado
- `renderizarHistorialPaginado()` — Renderiza historial con paginación
- `cambiarPaginaHistorial(direccion)` — Navega páginas del historial

Widget Actividades (Dashboard):

- `actualizarWidgetActividades()` — Actualiza widget en dashboard
- `completarActividadDesdeWidget(id)` — Completa desde widget
- `abrirModalDescripcionWidget(id)` — Abre modal para añadir nota
- `guardarDescripcionWidget(id)` — Guarda descripción extra
- `mostrarModalTodasActividadesOperador()` — Modal con todas las actividades (operador)
- `completarActividadDesdeModalOperador(id)` — Completa desde modal operador

Reportes Caracas:

- `mostrarBusqueda()` — Muestra sección de reportes
- `generarBusquedaCaracas()` — Genera reporte con filtros
- `renderizarTablaBusqueda()` — Renderiza tabla paginada
- `actualizarControlesPaginacionBusqueda()` — Crea controles de paginación
- `irAPaginaBusqueda(pagina)` — Navega páginas
- `calcularEstadoBusqueda(row)` — Calcula estado del cliente
- `renderizarGraficosBusqueda()` — Renderiza gráficos de barras y pastel
- `exportarBusquedaExcel()` — Exporta a Excel (xlsx)
- `exportarBusquedaPDF()` — Exporta a PDF con jsPDF
- `limpiarFiltrosBusqueda()` — Limpia todos los filtros

Reportes Maracaibo:

- `mostrarBusquedaMaracaibo()` — Muestra sección de reportes Maracaibo
- `generarBusquedaMaracaibo()` — Genera reporte con filtros
- `renderizarTablaBusquedaMB()` — Renderiza tabla paginada
- `actualizarControlesPaginacionBusquedaMB()` — Crea controles de paginación
- `irAPaginaBusquedaMB(pagina)` — Navega páginas
- `calcularEstadoBusquedaMB(row)` — Calcula estado del cliente
- `renderizarGraficosBusquedaMB()` — Renderiza gráficos
- `exportarBusquedaExcelMaracaibo()` — Exporta a Excel
- `exportarBusquedaPDFMaracaibo()` — Exporta a PDF
- `limpiarFiltrosBusquedaMaracaibo()` — Limpia filtros

Utilidades:

- `mostrarAlerta(mensaje, tipo)` — Muestra toast notification
- `formatearMoneda(valor)` — Formatea número a moneda
- `formatearFecha(dateString)` — Formatea fecha
- `getBadgeClass(estado)` — Devuelve clase CSS según estado
- `filtrarDeudores()` — Filtra tabla de deudores
- `verCliente(cedula)` — Navega a cliente específico
- `exportarReporte()` — Exporta CSV de estadísticas
- `ocultarMenuSegunRol()` — Oculta menús según rol
- `isAdmin()` — Verifica si es administrador
- `getUserRole()` — Obtiene rol del usuario
- `getTiendaUsuario()` — Obtiene tienda asignada

- `getUserIP()` — Obtiene IP del cliente

10.2 tienda-caracas-spa.js — Funciones Principales (61 funciones)

Inicialización:

- `initTiendaCaracas()` — Inicializa módulo Caracas
- `loadData()` — Carga datos desde API
- `processItemData(item)` — Procesa datos crudos (calcula deuda, depositado)

Base de Datos:

- `applyQuickFilter(filter)` — Aplica filtro rápido (todos/deudores/etc)
- `applyFilters()` — Aplica filtros avanzados
- `clearFilters()` — Limpia todos los filtros
- `debouncedFilter()` — Filtro con debounce (300ms)
- `updateSummary()` — Actualiza cards de resumen
- `updateFilterCounts()` — Actualiza contadores de filtros
- `renderTable()` — Renderiza tabla paginada
- `createRowHTML(item)` — Crea HTML de una fila
- `updatePagination()` — Actualiza controles de paginación
- `getTotalPages()` — Calcula total de páginas
- `goToPage(page)` — Navega a página
- `changeItemsPerPage()` — Cambia registros por página
- `getEstado(item)` — Determina estado del cliente
- `countCuotasPagadas(item)` — Cuenta cuotas pagadas

Modal Edición:

- `verDetalle(id)` — Abre modal de edición/ver
- `createModalElement()` — Crea HTML del modal dinámicamente
- `fillFormData(item)` — Llena formulario con datos del cliente
- `guardarCambios()` — Guarda cambios vía PUT
- `closeModal()` — Cierra modal
- `editarCliente(id)` — Alias de `verDetalle`

Eliminar Cuotas:

- `confirmarEliminarCuotas()` — Muestra confirmación con checklist
- `ejecutarEliminarCuotas(datosConfirmacion)` — Ejecuta eliminación vía PUT

Eliminar Cliente:

- `confirmarEliminarCliente(id)` — Modal de confirmación
- `eliminarCliente(id)` — Elimina vía DELETE

Conciliaciones:

- `mostrarConciliaciones()` — Muestra sección de conciliaciones
- `buscarFacturaConciliacion()` — Busca factura por número
- `mostrarClienteEncontrado(cliente)` — Muestra info del cliente
- `mostrarFormularioCuota(cliente)` — Muestra formulario nueva cuota
- `ocultarFormularioCuota()` — Oculta formulario
- `guardarCuotaConciliacion()` — Guarda nueva cuota vía PUT
- `volverABuscarFactura()` — Resetea a búsqueda inicial

Nuevo Registro:

- `mostrarFacturaNoEncontrada(nroFactura)` — Muestra mensaje no encontrada
- `mostrarFormularioNuevoRegistro()` — Muestra formulario nuevo cliente
- `mostrarNuevoRegistro(nroFactura)` — Prepara formulario con N° factura
- `guardarNuevaConciliacion()` — Guarda nuevo cliente vía POST

Tasa BCV:

- obtenerTasaPorFechaConciliacion() — Consulta tasa para cuota existente
- obtenerTasaNuevaConciliacion() — Consulta tasa para nueva cuota
- calcularDolarConciliacion() — Calcula dólar desde monto y tasa
- calcularDolarNuevaConciliacion() — Calcula dólar para nuevo registro

Historial:

- cargarHistorialCuotas(cliente) — Carga tabla de cuotas pagadas

Exportación:

- exportToExcel() — Exporta a CSV
- exportToPDF() — Muestra alerta (no implementado)
- printTable() — Imprime tabla
- downloadFile() — Descarga archivo generado

Navegación:

- mostrarBaseDatos() — Muestra base de datos
- mostrarMenuPrincipal() — Muestra menú principal

Modal Corporativo:

- mostrarModalCorporativo(titulo, mensaje, tipo, botones) — Modal genérico
- cerrarModalCorporativo() — Cierra modal genérico

Utilidades:

- formatCurrency(value) — Formatea moneda Bs
- formatNumber(value) — Formatea número con separador
- formatDate(dateString) — Formatea fecha
- showLoading(show) — Muestra/oculta loading
- parseNumberES(value) — Parsea número formato español

10.3 tienda-maracay-spa.js — Funciones (61 funciones, mismas con sufijo M)

Todas las funciones de tienda-caracas-spa.js pero con:

- Variables: allDataM, currentPageM, itemsPerPageM, etc.
- Funciones: initTiendaMaracay(), loadDataM(), applyFiltersM(), etc.
- API: /api/tienda-maracay en lugar de /api/tienda-caracas
- Envuelto en IIFE: (function(){...})()

10.4 tienda-maracaibo-spa.js — Funciones (61 funciones, mismas con sufijo MB)

Todas las funciones de tienda-caracas-spa.js pero con:

- Variables: allDataMB, currentPageMB, itemsPerPageMB, etc.
- Funciones: initTiendaMaracaibo(), loadDataMB(), applyFiltersMB(), etc.
- API: /api/tienda-maracaibo en lugar de /api/tienda-caracas
- Envuelto en IIFE: (function(){...})()
- Exposición de funciones al window global mediante objeto window
- Variable window.tiendaActiva para integración con estadísticas

11. HISTORIAL DE CAMBIOS Y EVOLUCIÓN

11.1 Versiones Anteriores

Versión	Fecha	Cambios Principales
v1-v3	2024-2025	Sistema base con IndexedDB local, sin backend
v4	2026-01	Migración a PostgreSQL + Node.js backend
v4.1	2026-02	Agregado autenticación JWT, roles admin/operador
v4.2	2026-03	Agregado módulo de actividades/agenda
v4.3	2026-04	Agregado reportes con filtros avanzados y exportación
v4.4	2026-05	Agregado Tienda Maracay (estructura idéntica a Caracas)
v4.5	2026-05	Agregado widget de actividades en dashboard, descripciones extra
v4.6	2026-05	Agregado eliminación de cuotas individuales, corrección de parseo numérico
v4.7	2026-06	Agregado conciliaciones con verificación de factura cancelada
v5	2026-07	Agregado Tienda Maracaibo, validación IP, auditoría, reportes Maracaibo, historial de actividades

des paginado

11.2 Bugs Corregidos Históricamente

#	Bug	Impacto	Solución
1	parseNumberES: "39.035,88" → 39.035	Perdía decimales en montos	Detectar último separador (coma vs punto)
2	monto_factura en edición se leía del input formateado	Perdía decimales al guardar	Usar valor original del objeto, no del input
3	Duplicado escapeHtml en app.js y auth.js	ReferenceError al cargar	setUtils() para pasar funciones entre módulos
4	IndexedDB borraba datos en onupgradeneeded	Pérdida total de datos	Crear store solo si no existe
5	Tasa BCV CDN fallaba y rompía la app	App no iniciaba	Fallback hardcodeado + try/catch aislado
6	app.js importaba EquiposPDF de archivo eliminado	Error de importación	Eliminar import obsoleto
7	database.js y equipos-database.js eliminaban store en upgrade	Pérdida de datos en upgrade	Condicional: crear solo si no existe
8	tasa-bcv.js tenía onclick inline con resolve de Promise	ReferenceError	Reemplazar con addEventListener y closure
9	Database no inicializaba después de login	Pantalla en blanco post-login	Extraer initPostLogin() que se ejecuta después de login exitoso
10	Event delegation no funcionaba en botones	Clics no respondían	Usar fase de captura y manejar data-section
11	Valores en reportes multiplicados ×100	32.400\$ en vez de 324\$	Corregir cálculo de total en imprimirVentaContado
12	Valores no centrados en tablas de reportes	Alineación incorrecta	Agregar text-align: center a celdas
13	Total Cancelado visible para vendedor	Filtración de información	Condicional: ocultar para rol vendedor
14	proxima_cuota mostraba 0 en TXT	Cálculo incorrecto	Trigger PostgreSQL con COALESCE y recálculo
15	Error 500 por columna duplicada proxima_cuota	Backend caído	Eliminar SELECT duplicado, usar SELECT *
16	App se congelaba al guardar crédito	Timeout/retry infinito	Non-blocking post-save flow en handleCreditoSubmit
17	Hybrid search no funcionaba en créditos	No se podía seleccionar cliente	Reemplazar select por input + dropdown
18	Carga TXT Créditos mostraba 0 Bs/USD	Datos incorrectos	Usar nuevos campos montoBsTotal, montoUsdTotal, tasaPromedio
19	GET /api/auth/users retornaba 500	No se podía listar usuarios	Corregir query en authController.js
20	Tienda Maracaibo era placeholder vacío	No había módulo	■ DESARROLLADO: tienda-maracaibo-spa.js completo

12. CHECKLIST PARA NUEVOS DESARROLLADORES

12.1 Para Agregar una Nueva Tienda (ej: Valencia)

1. **Backend:** Copiar endpoints de tienda-caracas en server.js y cambiar a tienda-valencia
2. **Base de Datos:** Crear tabla tienda_valencia con misma estructura
3. **Frontend:** Crear tienda-valencia-spa.js copiando tienda-caracas-spa.js y cambiar:
 - API_BASE_URL → /api/tienda-valencia
 - Variables: allDataV, currentPageV, initTiendaValencia(), etc.
4. **HTML:** Agregar sección contentValencia en panel.html
5. **CSS:** Agregar estilos en panel.css o crear tienda-valencia.css
6. **Navegación:** Agregar nav-valencia en sidebar
7. **Roles:** Verificar que operadores de Valencia vean solo esta tienda
8. **Reportes:** Agregar endpoint POST /api/reportes/valencia en reportes.js
9. **Estadísticas:** Agregar "valencia" en dropdown de tiendas en panel.js

12.2 Para Agregar un Nuevo Campo a Clientes

1. **Base de Datos:** Agregar columna a tienda_caracas, tienda_maracay y tienda_maracaibo
2. **Backend (server.js):** Agregar campo a allowedFields en POST y PUT de las 3 tiendas
3. **Frontend (SPA):** Agregar input en createModalElement() y fillFormData()
4. **Frontend (JS):** Agregar campo en el objeto data de guardarCambios()
5. **Tabla:** Agregar columna en createRowHTML()
6. **Filtros:** Agregar campo de filtro si aplica
7. **Reportes:** Agregar campo en filtros y exportación
8. **Estadísticas:** Verificar que no afecte cálculos de KPIs

12.3 Para Modificar el Cálculo de Deuda

1. **Backend:** Modificar lógica en server.js (endpoints PUT de tiendas)
2. **Frontend:** Modificar processItemData() en los 3 SPA (Caracas, Maracay, Maracaibo)
3. **Verificar:** Que monto_depositados y deuda se recalculen correctamente
4. **Estados:** Revisar getEstado() para que refleje nueva lógica
5. **Reportes:** Verificar que resumen de reportes use la nueva fórmula

12.4 Para Agregar un Nuevo Rol de Usuario

1. **Backend:** Agregar validación en routes/auth.js (login)
2. **Backend:** Agregar validación en routes/usuarios.js (crear/editar)
3. **Frontend:** Modificar isAdmin() en panel.js si el nuevo rol tiene permisos similares
4. **Frontend:** Modificar ocultarMenuSegunRol() para el nuevo rol
5. **Frontend:** Modificar mostrarSeccion() para validar permisos del nuevo rol
6. **Base de Datos:** Agregar restricción CHECK en columna rol si aplica

12.5 Para Agregar un Nuevo Módulo al Panel

1. **HTML:** Agregar content-area en panel.html con id único
2. **CSS:** Agregar estilos en panel.css
3. **JS:** Agregar función initNuevoModulo() en panel.js
4. **JS:** Agregar case en mostrarSeccion() para el nuevo módulo
5. **Sidebar:** Agregar nav-item en panel.html
6. **Roles:** Verificar permisos en mostrarSeccion()

7. **Backend:** Crear rutas API si es necesario

13. CONTACTO Y REFERENCIAS

Sistema: Créditos IPSFA C.A.
Stack: Node.js + Express + PostgreSQL + Vanilla JS
Base de datos: PostgreSQL (nombre: credits)
Puerto backend: 3000 (configurable por PORT en .env)
Autenticación: JWT en localStorage (clave: token)
Datos de usuario: localStorage (claves: usuario, tienda_usuario)
IP del cliente: Detectada automáticamente en login (req.ip)
Auditoría: Registro automático en tabla auditoria_usuarios para todas las acciones CRUD

13.1 Resumen de Archivos del Sistema v5

Archivo	Líneas	Funciones	Rol
server.js	600+	15	Servidor principal Express + endpoints inline
routes/auth.js	200+	5	Autenticación JWT + middleware
routes/bcv.js	150+	4	Tasas BCV (API externa)
routes/usuarios.js	400+	12	CRUD usuarios + auditoría
routes/actividades.js	300+	8	CRUD actividades + estadísticas
routes/estadisticas.js	250+	6	KPIs + gráficos
routes/reportes.js	200+	4	Reportes filtrados Caracas/Maracaibo
config/database.js	50+	2	Pool de conexión PostgreSQL
public/panel.html	800+	—	SPA principal (HTML)
public/js/panel.js	3,500+	94	Lógica principal SPA
public/js/tienda-caracas-spa.js	1,500+	61	Módulo Caracas
public/js/tienda-maracay-spa.js	1,500+	61	Módulo Maracay
public/js/tienda-maracaibo-spa.js	1,500+	61	Módulo Maracaibo
public/css/panel.css	1,800+	—	Estilos principales
public/css/tienda-caracas.css	800+	—	Estilos tienda Caracas

13.2 Métricas Finales del Sistema

Métrica	Valor
Total de Archivos Fuente	15+ archivos
Total de Líneas de Código	~13,000+ líneas
Total de Funciones	~343 funciones
Total de Endpoints API	28+ endpoints
Tiendas Soportadas	3 (Caracas, Maracay, Maracaibo)
Roles de Usuario	2 (Administrador, Operador)
Tablas de Base de Datos	6 (tienda_caracas, tienda_maracay, tienda_maracaibo, usuarios, actividades, aud
Módulos del Panel	10 secciones
Gráficos ApexCharts	2 (evolución mensual + distribución)

Exportaciones Soportadas	Excel (xlsx), PDF (jsPDF), CSV, Impresión
Actividades por Día	Ilimitadas (con historial paginado)
Auditoría	Automática para todas las acciones CRUD
Validación de IP	Obligatoria para operadores, opcional para admin

FIN DEL INFORME TÉCNICO

Documento generado el: **2026-07-16**

Total de archivos analizados: **15**

Total de líneas de código: **~13,000+**

Total de funciones: **~343**

Total de endpoints API: **28+**